Giolit Trading Risk Verifier

Version 1.0.0 — overview and installation

`trade-verify` proves that **every reachable trading behaviour** of a strategy and its risk controls stays within your limits – for every market move, fill, reject, cancel and operator action your assumptions allow – or shows the shortest event sequence that breaks a limit, as a FIX message log you can replay.

```
spec DeskControls;
units  { price = tick(0.05 INR); money = paise; }
ranges { price 20000 .. 40000; order_qty 1 .. 10000; }
mode controls_only;                                  // the controls against ANY strategy

strategy Any { parameters { max_long = 1000; max_short = 1000; max_clip = 500; } }

risk_controls {
    clip:      reject when order.qty > max_clip;
    long_cap:  reject when order.side == buy  and position + open_qty(buy)  + order.qty > max_long;
    short_cap: reject when order.side == sell and position - open_qty(sell) - order.qty < -max_short;
    kill:      reject when kill_switch;
}

constraints {
    long_limit:  always position <= max_long;
    short_limit: always position >= -max_short;
    order_size:  always on new_order: order.qty <= max_clip;
    kill_stops:  after kill_switch: never new_order;
    accounting:  always position == bought - sold;
}
```

```
$ trade-verify examples/06_controls_only.gts
  PROVEN      long_limit   always position <= max_long
              inductive lemmas [aggregate model: any number of orders], 0.03 s
  ...
Result: ALL CONSTRAINTS PROVEN (5 proven) in 0.6 s
```

What it does

Question (problem statement §9)	How
Q1 Can the system ever violate a constraint?	`trade-verify spec.gts`: PROVEN for every run and any number of orders, or VIOLATED with a run
Q2 Show me a violating sequence	the run, shortened, event by event; `--traces=DIR` or `--report=DIR` write it as text, JSON, **FIX 4.4 messages** and a market path
Q3 Is the strategy executable?	`feasible` targets in `constraints`: FEASIBLE with a run, or INFEASIBLE (proved)
Q4 What is the best/worst execution?	`objective { minimize implementation_shortfall; }` and `--optimize`: each extreme as a value a run reaches and a proved bound

Beyond the four questions:

- **Risk headroom** (`--headroom`): for every proven limit, the worst value any run reaches, how much of the limit is unused, and the Giolit Risk Resilience Score (1 to 5 stars).
- **Certificates** (`--report=DIR`): a PDF certificate or report, signed statement (full edition), proofs in your specification's terms, every run as FIX messages; `verify_certificate.py` lets an auditor check it offline.
- **Conformance replay** (`--conform=LOG`): replays a drop copy / execution-report log through the specification and reports every message the specification says cannot happen, every order the strategy sent that its rules would not send (or its controls would reject), and every constraint the observed run violates.
- **Imports**: client parameters from **FIXatdl** (verify every value a client can enter) and the venue's order state machine from **FIX Orchestra**.
- **Strategy pattern library**: a verified template per strategy family with its standard constraints (market maker, TWAP, VWAP, POV, pairs, lead-lag, delta hedge, smart order router, pre-trade risk gateway): `trade-verify --templates`.
- **Sanity checks** on every run: contradictory assumptions, rules that never fire, controls that never act.
- **Real numbers**: strategies written with `double` (IEEE binary64, exact counterexamples), e.g. a market maker with an EWMA volatility estimate.
- **Several venues and concurrency**: per-venue books, orders and limits, messages of different venues in any order, and risk checks on a thread of their own (`risk_controls async`) – the races single-venue testing never shows.
- **Several symbols and legs**: pairs, hedges and spreads – per-symbol books and positions, exposures (net, gross, beta-weighted), orders of several legs that pass the risk controls together, signals that take any value of their range (a model's output), and lags (`history(x, k)`): the legging and hedging failures a one-symbol test never shows.

Install

The package is self-contained (`code/bin/trade-verify`, `code/lib/*.so`, headers, examples). PDFs need LuaLaTeX (`texlive-luatex texlive-latex-extra texlive-fonts-extra`); password-protected PDFs need `qpdf`.

```
cd code && ./install.sh            # to ~/.local/giolit-trade-verifier, with trade-verify on your PATH
trade-verify --license             # trial: 1000 runs within 50 days
trade-verify share/giolit/examples/01_position_limit.gts
```

The examples

File	Shows
`01_position_limit.gts`	the problem statement's example: a check on the position alone lets two orders through (VIOLATED, 6 events)
`02_position_limit_fixed.gts`	counting open orders fixes it (PROVEN for any number of orders)
`03_kill_switch.gts`, `04_kill_switch_fixed.gts`	a fill handler re-enters after the kill switch; a control stops it
`05_late_fill.gts`	a venue that fills after a cancel: over-fill and a fill after cancel
`06_controls_only.gts`, `07_controls_short_gap.gts`	risk controls certified against any strategy; a gap on the sell side
`08_vwap.gts`, `09_vwap_fixed.gts`	a VWAP schedule that can never complete (INFEASIBLE, proved), and its fix
`10_vwap_fixatdl.gts`	client parameters from FIXatdl, the venue from FIX Orchestra: expiry + late fill over-executes

File	Shows
`11_execution_cost.gts`	optimization of the implementation shortfall
`12_avellaneda_stoikov.gts`, `13_..._fixed.gts`	the Avellaneda-Stoikov market maker in binary64: skewed quotes cross the market; clamped quotes are proved
`16_market_maker_pnl.gts`	what can be guaranteed about profit: the edge locked in and the worst-case loss are proved; “never under water” is violated
`14_smart_order_router.gts`, `15_..._fixed.gts`	a router on NSE and BSE: both venues fill the same quantity (overfill), a kill switch that stops one venue
`17_leg_hedger.gts`, `18_..._fixed.gts`	buy INFY, hedge each fill with TCS: a new clip before the hedge is done grows the unhedged quantity; waiting is proved
`19_pairs_legs.gts`, `20_..._fixed.gts`	a pairs trade with two-leg orders, a z-score signal and a lag: an exchange reject of one leg leaves the pair half done; opening only from balanced positions is proved (with exposure limits)
`logs/*.fix`	logs for `--conform`

The Tutorial teaches the language and every option; `DESIGN.md` explains how it works.

Exit codes

0 every constraint proven (and every target feasible); 1 a constraint is violated; 2 some undecided or bounded; 3 usage or input error; 4 no valid license. With `--conform`: 0 the log conforms, 1 it does not.

Giolit Labs, Thrissur, Kerala, India – www.giolit.com – business.giolitlabs@gmail.com